Domain-Driven Design Aggregates, Domain Events, and Value Objects

@VaughnVernon

Copyright © 2021 Vaughn Vernon. All rights reserved.

Pearson

DDD Modeling

Domain-Driven Design offers both strategic and tactical modeling tools. Today's course provides a brief overview of DDD strategic modeling, and takes you into depth on tactical modeling.

Goals

What you'll learn-and how you can apply it

The range of DDD tactical modeling tools supporting a Ubiquitous Language, including Aggregates, Domain Events, and Value Objects in depth

Effective design of Aggregates reflecting the Ubiquitous Language and guided by the "Rules of Aggregate Design," as well as testing Aggregates

Domain Events, Event Sourcing for Aggregate persistence, and Event-Driven Architecture

Value Objects that reflect the Ubiquitous Language, support immutability with side-effect-free behavior, and how they enhance Aggregate design.

Schedule

Segment 1: DDD Tactical Modeling

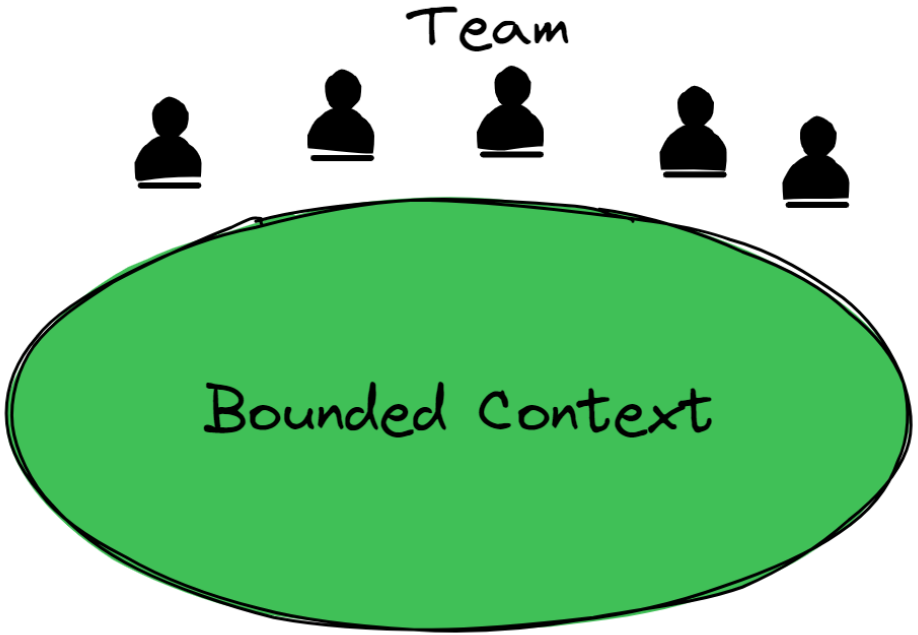
Overview of Bounded Context, Ubiquitous Language, and Tactical Tools/Patterns

Effective Aggregate Design: Applying the Rules

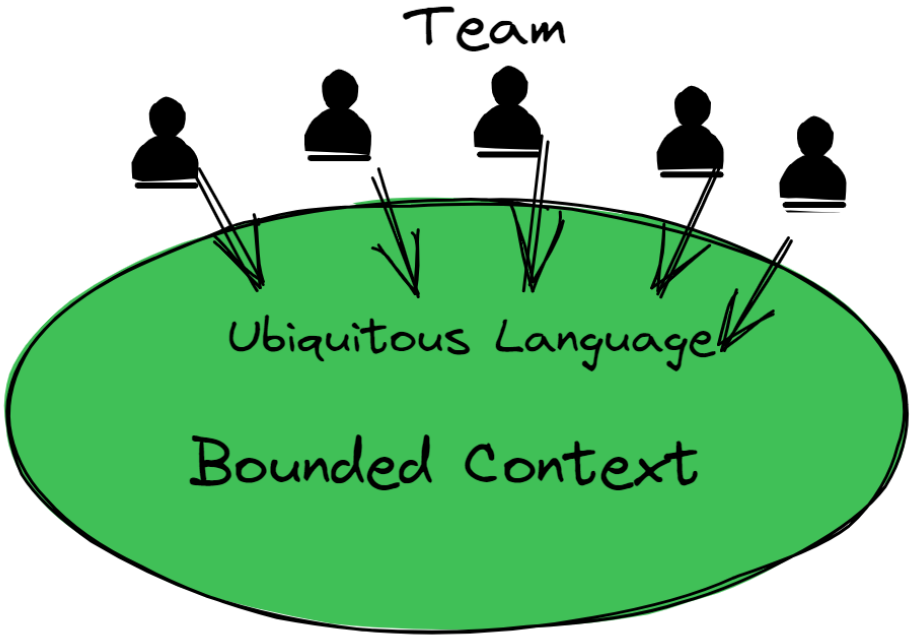
Modeling Concepts as Value Objects: Power Through Simplicity

Exercise: Identify a Ubiquitous Language in your domain; recite the Rules of Aggregate Design; explain the advantages of using Value Objects

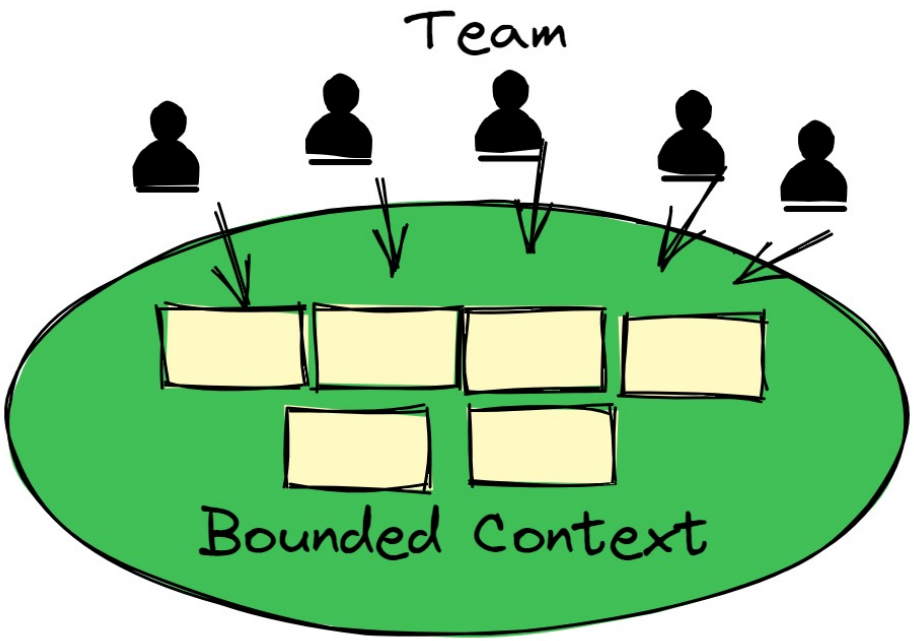
Bounded Context



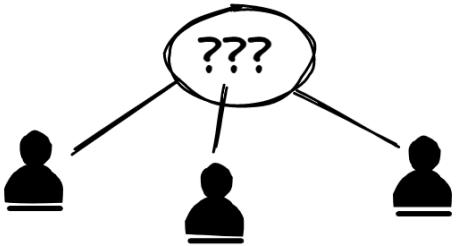
Ubiquitous Language

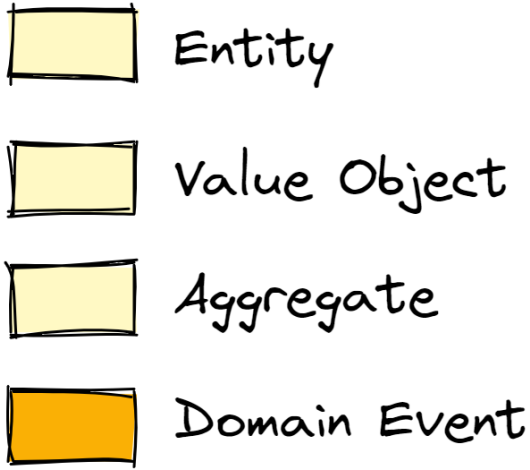


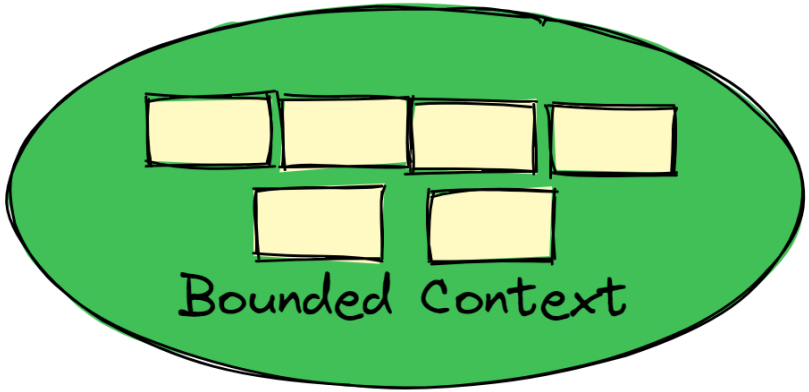
Ubiquitous Language as Model



Tactical Tools / Patterns







Tactical Tools Definitions

Entity: uniquely identified model concept; mutable

Value Object: Functional model concept; immutable

Aggregate: Entity graph with transactional scope

Domain Event: Record of notable happening; immutable

Exercise

Identify a Ubiquitous Language in your domain

Recite the Rules of Aggregate Design

Explain the advantages of using Value Objects

Schedule

Segment 2: Designing Aggregates and Domain Events

Business Invariants and Aggregate Consistency Boundaries

Designing a Traditional Object-Oriented Aggregate with Commands and Queries

What Are Domain Events and How Are They Modeled?

Exercise: Describe business invariants and consistency boundaries; explain how traditional object-oriented Aggregates are designed; design a Domain Event and how emitted from an Aggregate

Aggregate Rules of Thumb: 1

Protect business invariants inside consistency boundaries

What is a business invariant?

What is an Aggregate consistency boundary?

How is a consistency boundary modeled?

Aggregate Rules of Thumb: 2

Design small Aggregates

Why is small best?

What is “small” and when is it small enough?

How to design “traditional” object-oriented Aggregate?

Aggregate Rules of Thumb: 3

Reference other Aggregates by identity only

Why not design with direct object references?

How does reference by identity work?

What are the advantages of reference by identity?

Aggregate Rules of Thumb: 4

Use eventual consistency for updates outside a single Aggregate boundary

Why not update multiple Aggregates in one transaction?

What are my options for eventual consistency?

How does contemporary eventual consistency work?

Domain Events

Domain Events are used prolifically in DDD models

What is a Domain Event?

How do I model an Aggregate to use Domain Events?

What are my options for persistence and publishing?

Exercise

Describe by example:

Business invariants and consistency boundaries

Explain how traditional object-oriented Aggregates are designed

Design a Domain Event and how emitted from an Aggregate

Schedule

Segment 3: Modeling Value Objects

Immutability and Side-Effect-Free Behavior

Objects with Functions

Reflecting a Ubiquitous Language as Conceptual Whole Values and Fluent Behavior

Exercise: Explain immutability, side-effect-free behavior, and functions; explain how to use fluent behavior to express a Ubiquitous Language

Value Immutability

Values are values are values

You can’t change the value of 1, or 2, or 100, or ...

Values always have identity but it’s not unique

How do I design for immutability (see next)

Side-Effect-Free Behavior

Since Values can’t be mutated / changed...

All behavior must be pure functions

Pure functions have no side effects on current state

Values have unchanging equality and hash codes

Values are replaceable

Values and Ubiquitous Language

Prefer Value Objects over Entities whenever possible

Values can be fluent language expressions

Values expressions are pure functions

Aggregate can be only one Value state

- Aggregate state as one Value makes for simple tests

Exercise

Explain by example:

Immutability

Side-effect-free behavior

Functions

Fluent expressions as behavior

Schedule

Segment 4: Event Sourcing + Event-Driven Architecture

What Is Event Sourcing and Why Is It Useful?

Aggregate Design with Command-Centric Behavior and Event Sourcing

What Is Event-Driven Architecture?

Exercise: Describe Event Sourcing and why it might be useful in certain domains; explain how Event Sourced Aggregate is different from a traditional object design; define Event-Driven Architecture with Domain Events

Event Sourcing

Explain Event Sourcing

What is Event Sourcing and why is it useful?

Command-centric behavior Aggregate design

Applying events, twice, etc.

Event Sourcing

Explain Event Sourcing

What is Event Sourcing and why is it useful?

Command-centric behavior Aggregate design

Applying events, twice, etc.

Event-Driven Archtecture

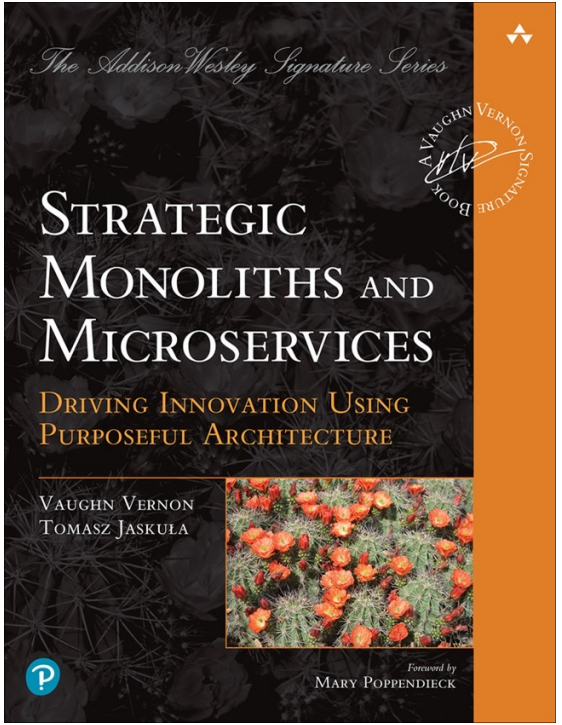
Explain Event-Driven Architecture

Publishing Domain Events

Subscribing to Domain Events

Eventual Consistency

Strategic Monoliths and Microservices



Upcoming Live-Virtual Workshops

September 18-21: Kalele.io

Implementing Domain-Driven Design

Most popular; taught to thousands

https://kalele.io/training/iddd/

Thanks!

Domain-Driven Design Aggregates, Domain Events, and Value Objects

@VaughnVernon

Copyright © 2020 Vaughn Vernon. All rights reserved.

Pearson